

Skip – List
–Delete Element
–Program Analysis – Type B

Program Analysis:

```
void deleteElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Find the node, keeping track of pointers that point to it
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    current = current->forward[0];

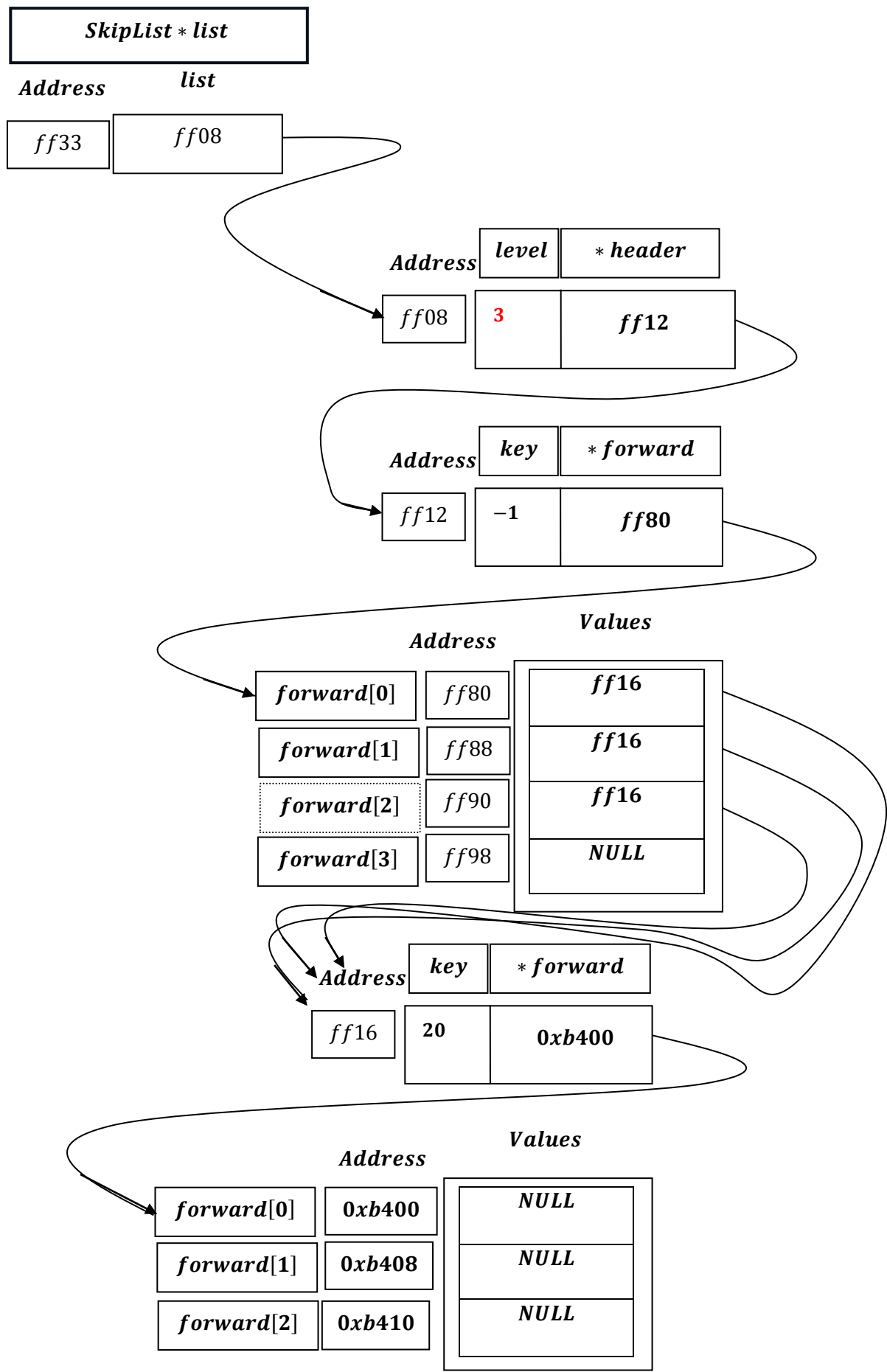
    // If the target node is found
    if (current != NULL && current->key == key) {
        // Rewire the pointers to bypass the deleted node
        for (int i = 0; i <= lst->level; i++) {
            if (update[i]->forward[i] != current) {
                break; // If a level doesn't point to current, higher levels
won't
                // either
            }
            update[i]->forward[i] = current->forward[i];
        }

        // Free memory using free()
        free(current->forward);
        free(current);

        // Reduce the maximum level of the list if layers become empty
        while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
```

```
        lst->level--;  
    }  
    cout << "Successfully deleted key " << key << endl;  
} else {  
    cout << "Key " << key << " not found. Cannot delete." << endl;  
}  
}  
  
... .  
case 2:  
    cout << "Enter key to delete: ";  
    cin >> key;  
    deleteElement(list, key);  
    break;
```

B. Now Say ,We have single Node left ,after deletion.



```
deleteElement(list, key);
```

deleteElement(list: ff08, key: 20) → passed .

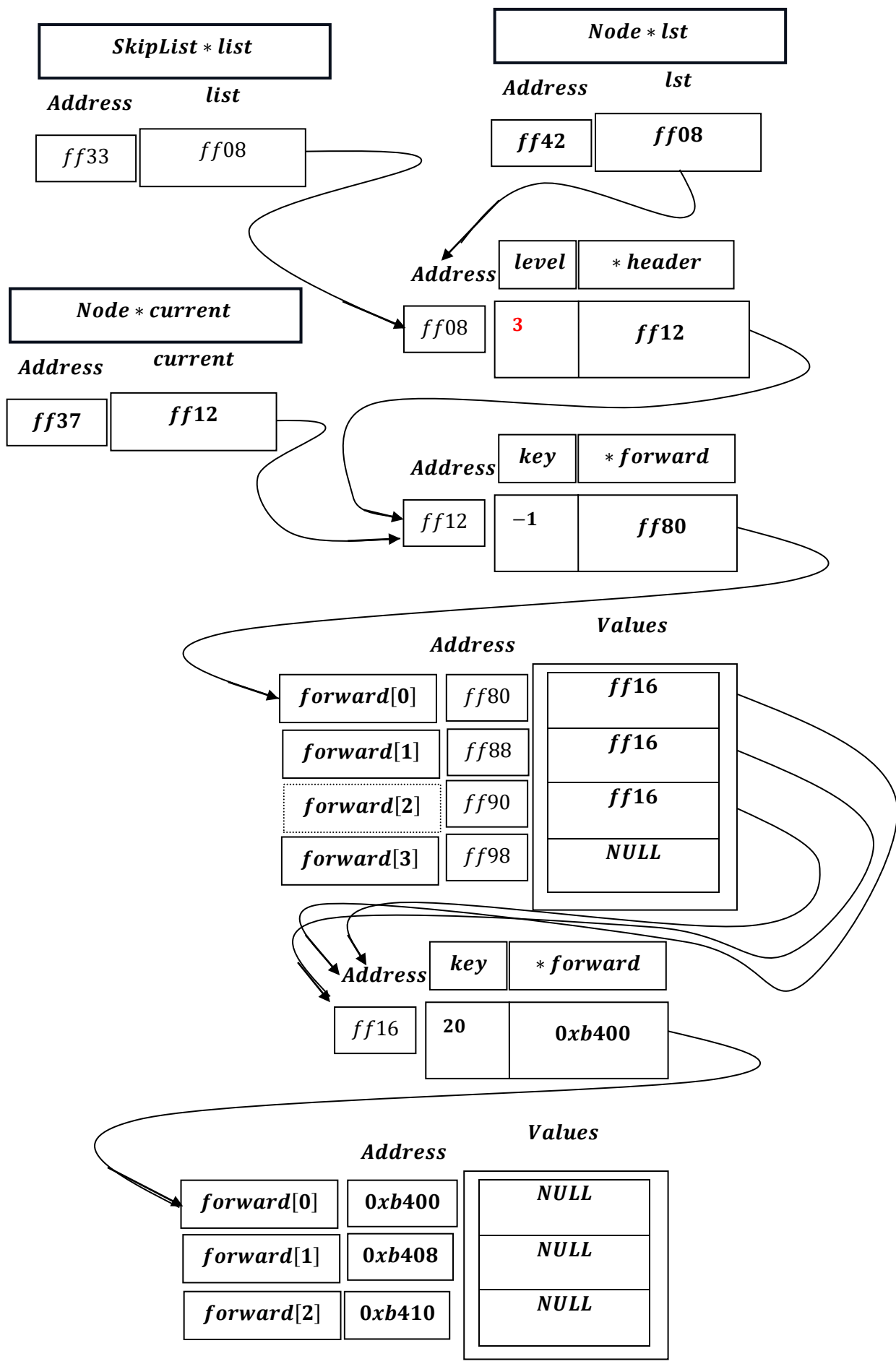
And,

```
void deleteElement(struct SkipList *lst, int key)
```

deleteElement(list := lst: ff08, key := key: 20) → Assigned

```
struct Node *current = lst->header;
```

current = lst → header = ff12.



```
struct Node *update[MAX_LEVEL + 1];
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4],

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

Array Element	Hypothetical Stack Address	Value (Garbage Data)	Pointer Status
update[0]	0x7fff0000 to 0x7fff0007	0x4a3f8921bce45099	Wild Pointer (Points to a random, invalid address)
update[1]	0x7fff0008 to 0x7fff000f	0x0000000000000002	Wild Pointer
update[2]	0x7fff0010 to 0x7fff0017	0x7ffee1245a00881c	Wild Pointer
update[3]	0x7fff0018 to 0x7fff001f	0x0000000000000000	Accidentally NULL (Just by chance)

More simpler way:

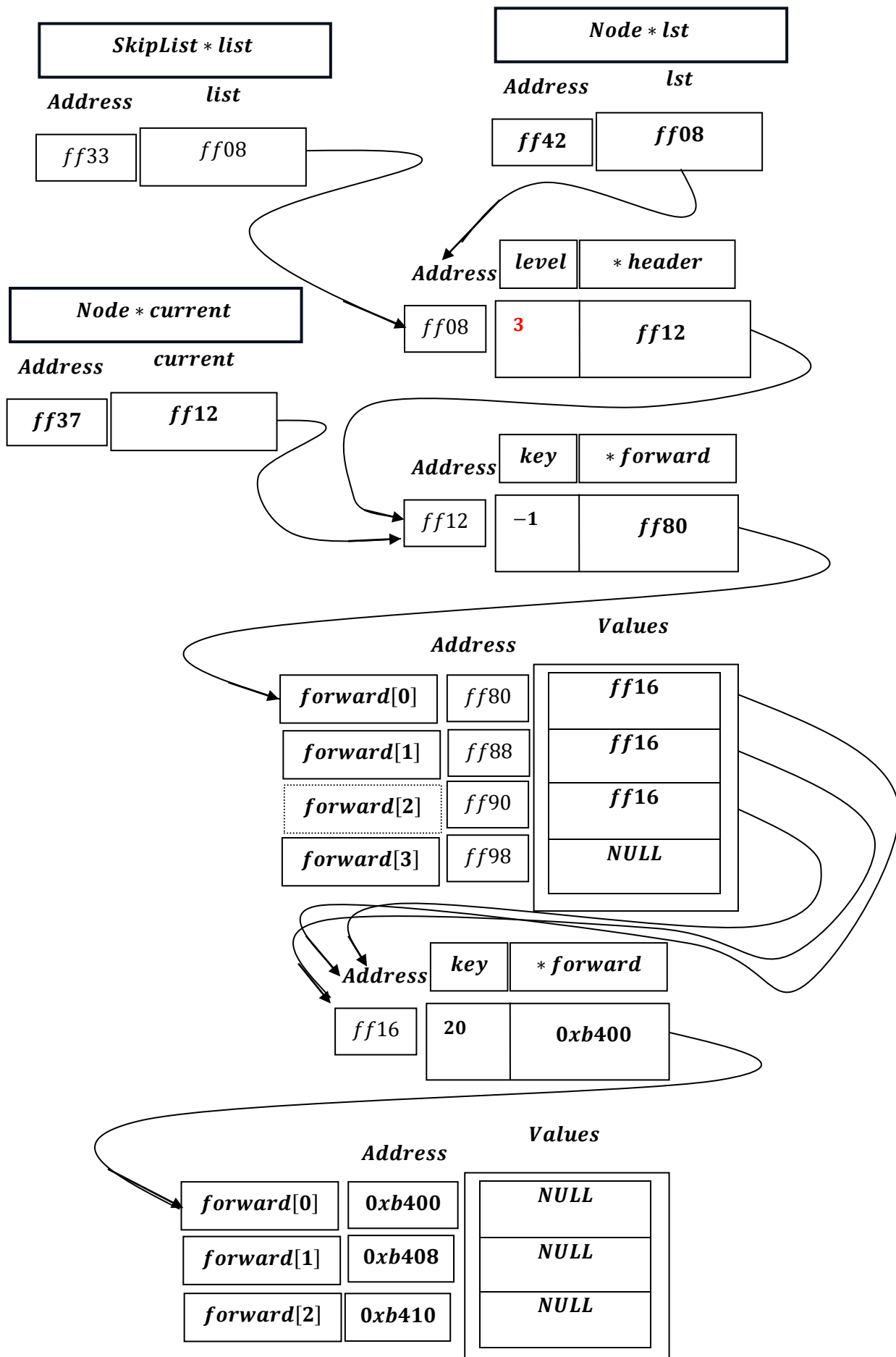
	Address	Values	
update[0]	0x7fff0000	0x4a3f8921bce45099	[8 bytes]
update[1]	0x7fff0008	0x0000000000000002	[8 bytes]
update[2]	0x7fff0010	0x7ffee1245a00881c	[8 bytes]
update[3]	0x7fff0018	0x0000000000000000	[8 bytes]

```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

``memset`` stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

Here we kept addresses same like: 0x7fff0000 , just for an example , but it may not be same , in reality.



	<i>Address</i>	<i>Values</i>
<i>update</i> [0]	0x7fff0000	<i>NULL</i>
<i>update</i> [1]	0x7fff0008	<i>NULL</i>
<i>update</i> [2]	0x7fff0010	<i>NULL</i>
<i>update</i> [3]	0x7fff0018	<i>NULL</i>

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

for: int i = lst → level: 3; i ≥ 0;

while: current: ff12 → forward[3]: NULL ≠ NULL $\left[\begin{array}{l} \text{False,} \\ \text{Loop Ends} \end{array} \right]$

update[3] = current = ff12;

i = i − 1 = 3 − 1 = 2.

for: int i = 2; i ≥ 0;

while: current: ff12 → forward[2]: ff16 ≠ NULL[True] AND

current: ff12 → forward[2]: ff16 → key: 20 < key: 20 $\left[\begin{array}{l} \text{False,} \\ \text{Loop Ends} \end{array} \right]$

update[2] = current = ff12;

i = i − 1 = 2 − 1 = 1.

for: int i = 1; i ≥ 0;

while: current: ff12 → forward[1]: ff16 ≠ NULL[True] AND

current: ff12 → forward[1]: ff16 → key: 20 < key: 20 $\left[\begin{array}{l} \text{False,} \\ \text{Loop Ends} \end{array} \right]$

update[1] = current = ff12;

i = i - 1 = 1 - 1 = 0.

for: int i = 0; i ≥ 0;

while: current: ff12 → forward[0]: ff16 ≠ NULL[True] AND

current: ff12 → forward[0]: ff16 → key: 20 < key: 20 $\left[\begin{array}{l} \text{False,} \\ \text{Loop Ends} \end{array} \right]$

update[0] = current = ff12;

i = i - 1 = 0 - 1 = -1.

for: int i = -1; i ≥ 0; $\left[\begin{array}{l} \text{False,} \\ \text{Loop Ends} \end{array} \right]$

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Node * current*

Address current

ff37 ff12

Address level * header

ff08 3 ff12

Address key * forward

ff12 -1 ff80

Address

Values

forward[0]	ff80	ff16
forward[1]	ff88	ff16
forward[2]	ff90	ff16
forward[3]	ff98	NULL

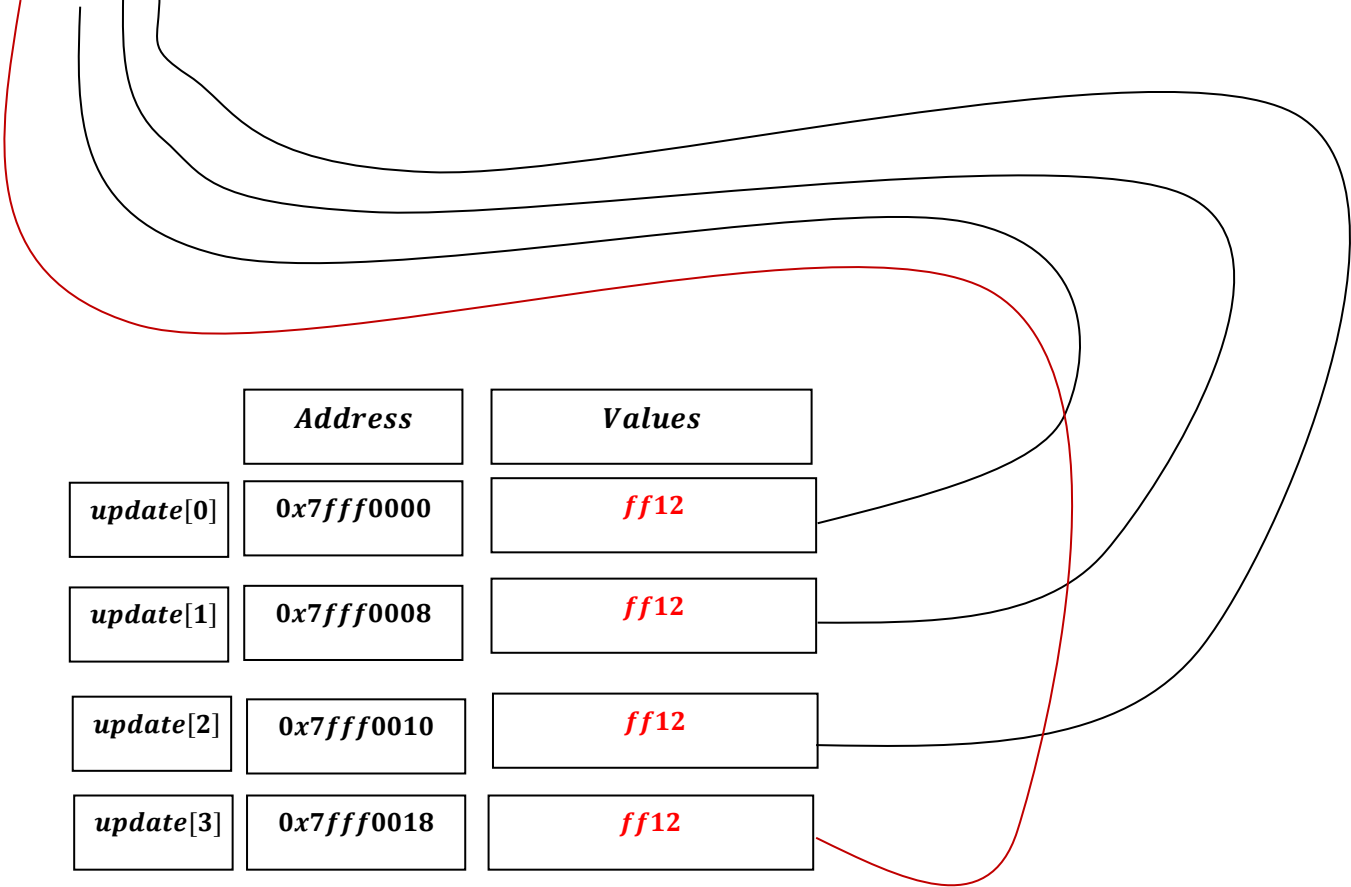
Address key * forward

ff16 20 0xb400

Address

Values

forward[0]	0xb400	NULL
forward[1]	0xb408	NULL
forward[2]	0xb410	NULL



	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	0x7fff0000	<i>ff12</i>
<i>update[1]</i>	0x7fff0008	<i>ff12</i>
<i>update[2]</i>	0x7fff0010	<i>ff12</i>
<i>update[3]</i>	0x7fff0018	<i>ff12</i>

```
current = current->forward[0];
```

current = *current: ff12* → *forward[0]: ff16*

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Node * current*

Address current

ff37 ff16

Address level * header

ff08 3 ff12

Address key * forward

ff12 -1 ff80

Address

Values

forward[0]	ff80	ff16
forward[1]	ff88	ff16
forward[2]	ff90	ff16
forward[3]	ff98	NULL

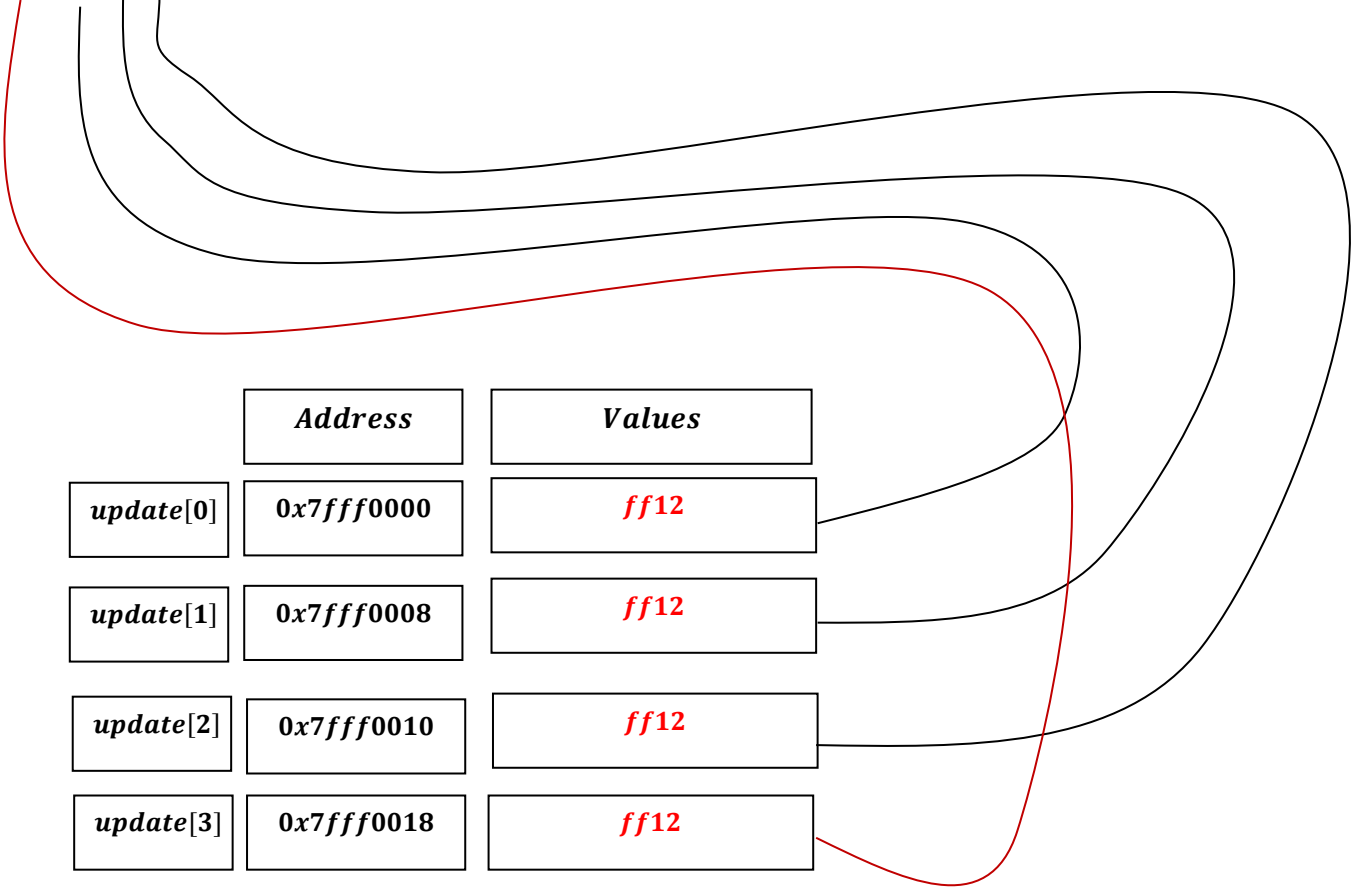
Address key * forward

ff16 20 0xb400

Address

Values

forward[0]	0xb400	NULL
forward[1]	0xb408	NULL
forward[2]	0xb410	NULL



	Address	Values
update[0]	0x7fff0000	ff12
update[1]	0x7fff0008	ff12
update[2]	0x7fff0010	ff12
update[3]	0x7fff0018	ff12

```
if (current != NULL && current->key == key) {
```

current: ff16 $\neq$ NULL AND current: ff16 $\rightarrow$ key: 20 == 20:

```
for (int i = 0; i <= lst->level; i++) {
    if (update[i]->forward[i] != current) {
        break; // If a level doesn't point to current, higher levels
won't
        // either
    }
    update[i]->forward[i] = current->forward[i];
}
```

int i = 0 ; i $\leq$ lst $\rightarrow$ level: 3

if(update[0]: ff12 $\rightarrow$ forward[0]: ff16 $\neq$ current: ff16) ***Condition***
is
False

update[0] $\rightarrow$ forward[0] = current: ff16 $\rightarrow$ forward[0]: NULL

i = i + 1 = 0 + 1 = 1.

int i = 1 ; i ≤ lst → level: 3

if(update[1]: ff12 → forward[1]: ff16 ≠ current: ff16) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

update[1] → forward[1] = current: ff16 → forward[1]: NULL

i = i + 1 = 1 + 1 = 2.

int i = 2 ; i ≤ lst → level: 3

if(update[2]: ff12 → forward[2]: ff16 ≠ current: ff16) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

update[2] → forward[2] = current: ff16 → forward[2]: NULL

i = i + 1 = 2 + 1 = 3.

int i = 3 ; i ≤ lst → level: 3

if(update[3]: ff12 → forward[3]: NULL ≠ current: ff16) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

break; $\left[\begin{array}{c} \text{The control flow statement break;} \\ \text{Terminates loop immediately terminates} \\ \text{here.} \end{array} \right]$

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Node * current*

Address current

ff37 ff16

Address

level * header

ff08 3 ff12

Address

key * forward

ff12 -1 ff80

Address

Values

forward[0]	ff80	NULL
forward[1]	ff88	NULL
forward[2]	ff90	NULL
forward[3]	ff98	NULL

Address

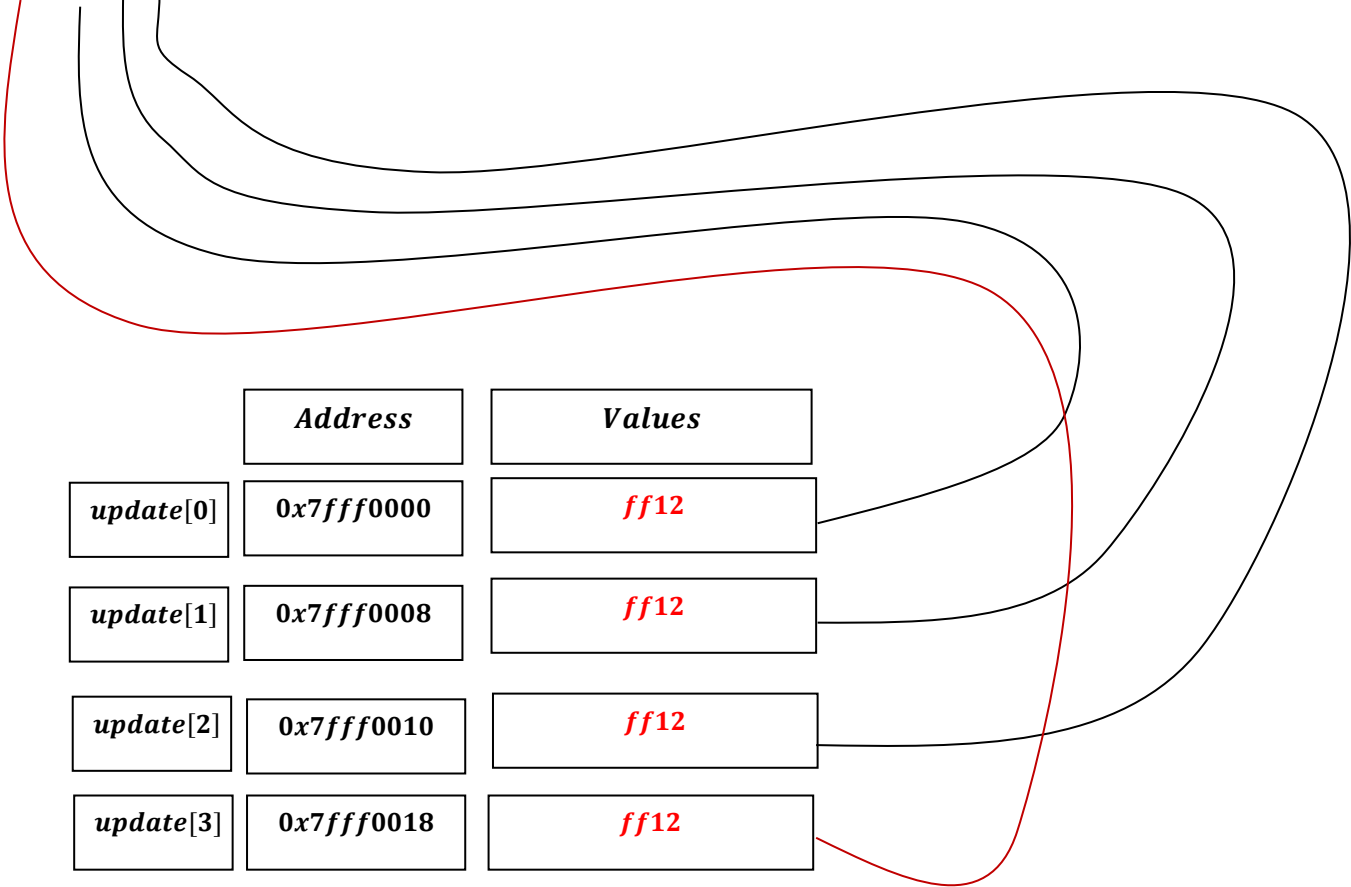
key * forward

ff16 20 0xb400

Address

Values

forward[0]	0xb400	NULL
forward[1]	0xb408	NULL
forward[2]	0xb410	NULL



	Address	Values
update[0]	0x7fff0000	ff12
update[1]	0x7fff0008	ff12
update[2]	0x7fff0010	ff12
update[3]	0x7fff0018	ff12

```
free(current->forward);
free(current);
```

free(current: ff16 → forward)

The pointer ***current → forward*** holds the starting address of this chunk, which is **0xb400**. This is where ***forward[0]*** lives.

The Offset: The address **0xb408** is simply the location of the second element in that array (***forward[1]***). It is exactly 8 bytes after the starting address, sitting inside the same block of memory.

When the program executes ***free(current → forward)***, it tells the system to deallocate the entire chunk of memory that begins at **0xb400**. Because **0xb408** is just a part of that continuous chunk, it gets completely freed at the exact same time.

It will not add any offsets and delete each block of memory but all at once.

And ,

```
free(current);
```

Will free the memory block that current pointer pointed to.

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Node * current*

Address current

ff37 ff16

Address

*level * header*

ff08 3 ff12

Address

*key * forward*

ff12 -1 ff80

Address

Values

<i>forward[0]</i>	<i>ff80</i>	<i>NULL</i>
<i>forward[1]</i>	<i>ff88</i>	<i>NULL</i>
<i>forward[2]</i>	<i>ff90</i>	<i>NULL</i>
<i>forward[3]</i>	<i>ff98</i>	<i>NULL</i>

Address

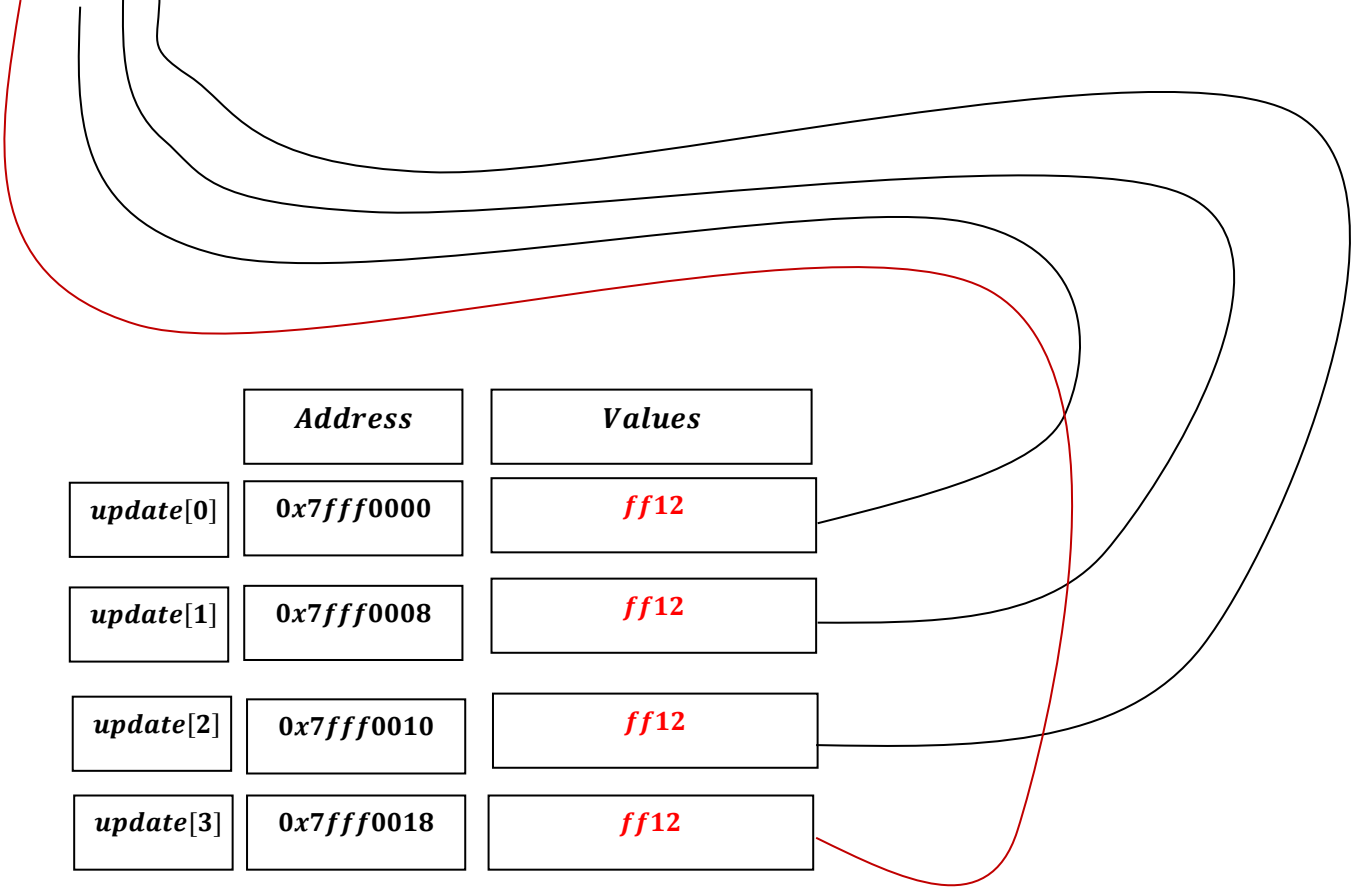
Value

ff16

Address

Values

<i>0xb400</i>	
<i>0xb408</i>	
<i>0xb410</i>	



	Address	Values
update[0]	0x7fff0000	ff12
update[1]	0x7fff0008	ff12
update[2]	0x7fff0010	ff12
update[3]	0x7fff0018	ff12

```

while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
    lst->level--;
}
cout << "Successfully deleted key " << key << endl;
}

```

Now layers becomed **Empty**.

$lst \rightarrow level: 3 > 0$ AND $lst \rightarrow header: ff12 \rightarrow forward[lst \rightarrow level: 3] NULL == NULL$:
 $lst \rightarrow level = lst \rightarrow level - 1 = 3 - 1 = 2$

$lst \rightarrow level: 2 > 0$ AND $lst \rightarrow header: ff12 \rightarrow forward[lst \rightarrow level: 2] NULL == NULL$:
 $lst \rightarrow level = lst \rightarrow level - 1 = 2 - 1 = 1$

$lst \rightarrow level: 1 > 0$ AND $lst \rightarrow header: ff12 \rightarrow forward[lst \rightarrow level: 1] NULL == NULL$:
 $lst \rightarrow level = lst \rightarrow level - 1 = 1 - 1 = 0$

$lst \rightarrow level: 0 > 0$ Condition
becomes
False

*SkipList * list*

Address list

ff33 ff08

*Node * lst*

Address lst

ff42 ff08

*Node * current*

Address current

ff37 ff16

Address

level * header

ff08 0 ff12

Address

key * forward

ff12 -1 ff80

Address

Values

forward[0]	ff80	NULL
forward[1]	ff88	NULL
forward[2]	ff90	NULL
forward[3]	ff98	NULL

Address

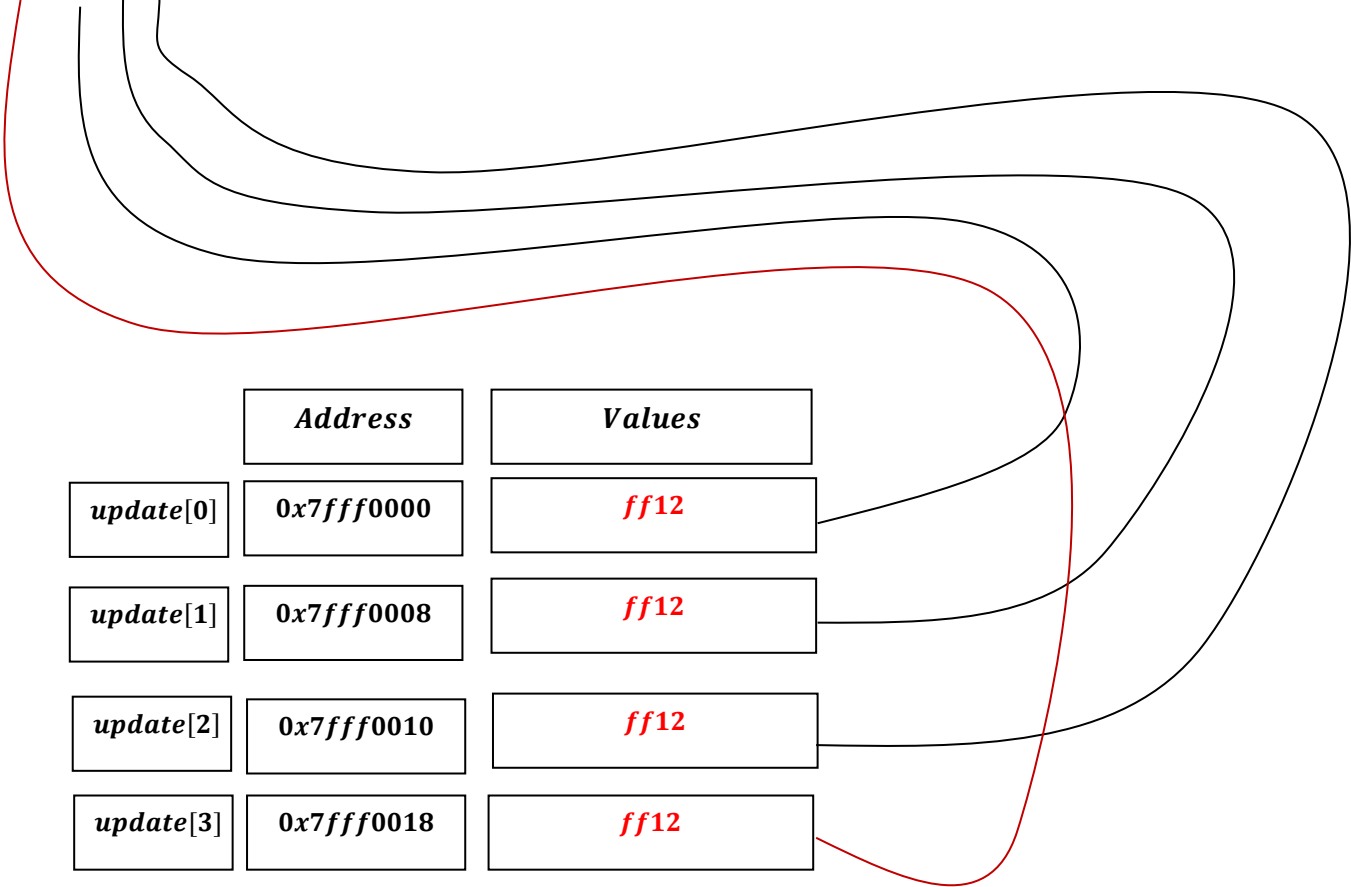
Value

ff16

Address

Values

0xb400	
0xb408	
0xb410	



The diagram illustrates a memory structure with four rows. Each row contains three boxes: a label box on the left, an 'Address' box in the middle, and a 'Values' box on the right. The labels are `update[0]`, `update[1]`, `update[2]`, and `update[3]`. The addresses are `0x7fff0000`, `0x7fff0008`, `0x7fff0010`, and `0x7fff0018`. The values are all `ff12` in red. A red line starts from the top left, curves around the top, and then points to the 'Values' box of the `update[3]` row, indicating a deletion operation.

	Address	Values
<code>update[0]</code>	<code>0x7fff0000</code>	<code>ff12</code>
<code>update[1]</code>	<code>0x7fff0008</code>	<code>ff12</code>
<code>update[2]</code>	<code>0x7fff0010</code>	<code>ff12</code>
<code>update[3]</code>	<code>0x7fff0018</code>	<code>ff12</code>

```
cout << "Successfully deleted key " << key << endl;
```

Now , it prints ``Successfully deleted key : 20.``

After function finishes , temporary variables get destroyed:

Node * current

Address current

ff37 ff16

Destroyed

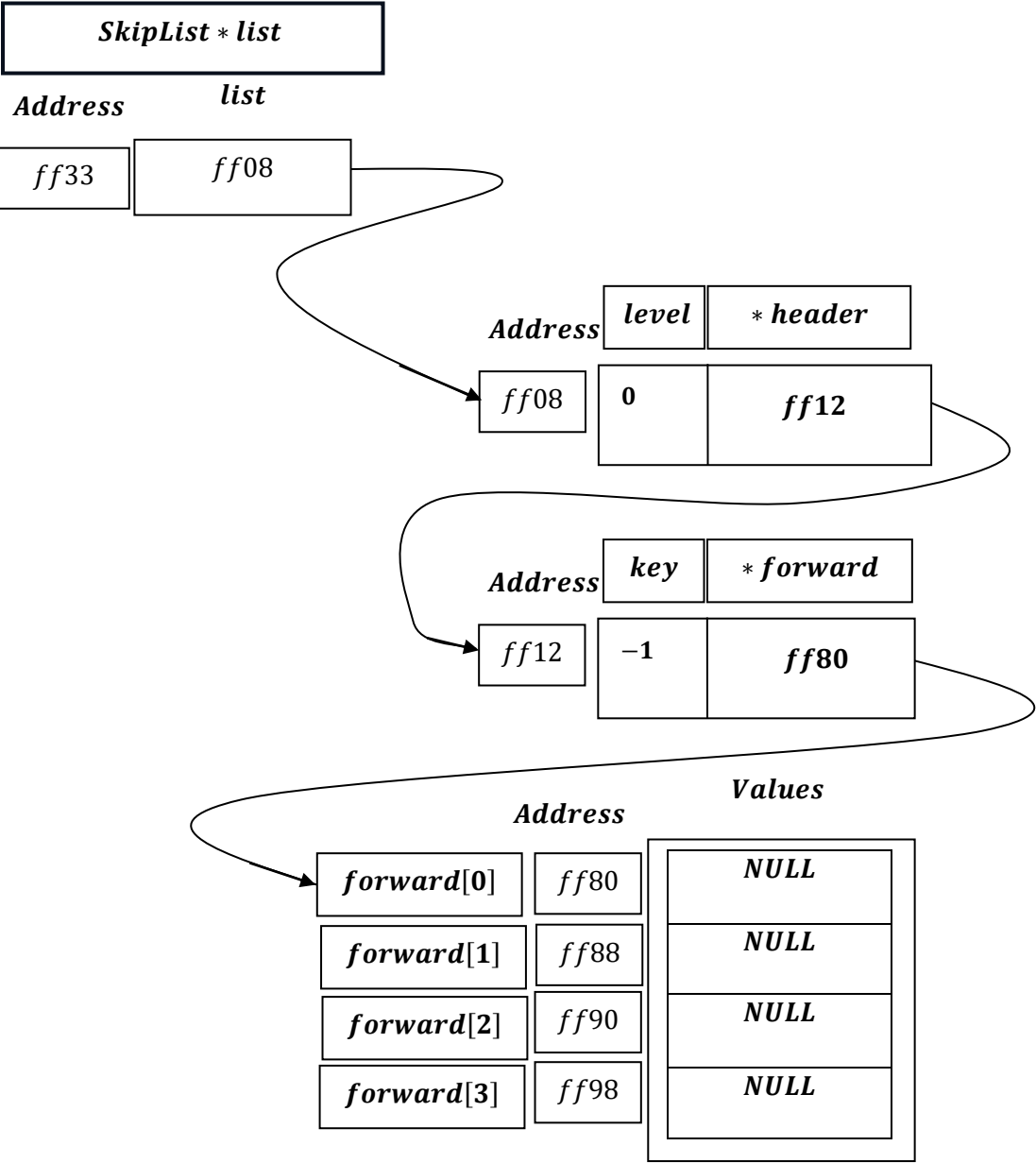
	Address	Values
update[0]	0x7fff0000	ff12
update[1]	0x7fff0008	ff12
update[2]	0x7fff0010	ff12
update[3]	0x7fff0018	ff12

Node * lst

Address lst

ff42 ff08

And we are left with:



Type C [*When Key is*
Not Found]

This we can interpret the process. By your own.
